CO528

Mini Project

Prepared by :

Team BitNex

DEPARTMENT OF COMPUTER ENGINEERING
UNIVERSITY OF PERADENIYA

CO528 – Applied Software Architecture

Department Engagement & Career Platform (DECP)

Project Report

Course	CO528 – Applied Software Architecture
Project	Mini Project – Department Engagement & Career Platform
Module Code	DECP
Architecture Style	Cloud-Native Kafka Microservices Architecture
Team Name	BitNex
Team Members	E/20/032 – Bandara A.M.N.C. E/20/157 – Janakantha S.M.B.G. E/20/254 – Methpura S.K.P.

Project Roles

Role	Responsibility
Enterprise Architect	High-level enterprise integration and workflow design
Solution Architect	End-to-end solution design across all services
Application Architect	Service-level design, APIs, and module communication
Security Architect	Authentication, authorization, and data security strategy
DevOps Architect	Cloud deployment, CI/CD, containerization, and observability

Powered by Cloud-Native Microservices | Apache Kafka | NestJS | React | Docker

Table of Contents

1. Executive Summary 5
2. Project Overview & Objectives 5
2.1 Objective 5
2.2 Scope 6
3. Functional Requirements 6
4. Non-Functional Requirements (Quality Attributes) 7
5. Architecture Design 7
5.1 Enterprise Architecture 7
5.2 Service-Oriented Architecture (SOA) 8
5.3 Event-Driven Architecture (Apache Kafka) 8
5.3.1 Kafka Topic Design 9
5.3.2 Standard Event Schema 9
5.3.3 Example Event Flow: Job Application 9
5.4 Communication Model 10
5.5 Web-Oriented Architecture 10
5.6 Mobile Architecture 10
6. Product Architecture 11
6.1 Modular Design 11
6.2 Reusable Components 11
7. Technology Stack 12
8. Implementation Details 13
8.1 Identity Service 13
8.1.1 Database Schema (PostgreSQL) 13
8.1.2 Redis Usage 13
8.1.3 Authentication Flow 13
8.1.4 Role-Based Access Control (RBAC) 14
8.1.5 Bulk User Provisioning 14
8.1.6 Kong API Gateway Integration 14
8.2 Engagement Service 14
8.2.1 Core Features 14
8.2.2 MongoDB Collections 15
8.2.3 Kafka Events Published 15
8.3 Career Service 15
8.3.1 Domain Models 15
8.3.2 Access Control Rules 16

8.3.3 Resume Upload Strategy 16
8.4 Event Service 16
8.5 Notification Service 17
8.5.1 Kafka Topics Consumed 17
8.5.2 Notification Channels 17
8.5.3 Idempotency Protection 17
8.6 Analytics Service 17
8.7 Messaging Service 18
9. Infrastructure & Cloud Deployment 18
9.1 Local Development Environment (Docker Compose) 18
9.2 Cloud Deployment Architecture (AWS) 19
9.2.1 Scalability Considerations 20
10. Observability & Production Readiness 20
10.1 Health & Readiness Probes 20
10.2 Metrics (Prometheus + Grafana) 20
10.3 Centralised Logging (ELK Stack) 20
10.4 Distributed Tracing (OpenTelemetry) 21
10.5 Error Handling Strategy 21
11. Security Architecture 21
12. Research Findings: Platform Analysis & Improvement Proposals 22
12.1 Analysis of Real-World Platforms 22
12.2 Feature Gaps Identified 22
12.3 Architectural Improvements Proposed 23
12.3.1 AI Integration Roadmap 23
12.3.2 Schema Registry for Kafka 23
12.3.3 Service Mesh 23
13. CI/CD & DevOps Strategy 23
14. Architecture Diagrams 24
14.1 Enterprise Architecture Diagram 24
14.2 SOA/Microservice Architecture Diagram 25
14.3 Product Modularity Diagram 26
14.4 Deployment Architecture Diagram 27
15. Conclusion 28
16. Appendix: Reference Links 28

1. Executive Summary

The Department Engagement & Career Platform (DECP) is a university-focused social and professional networking platform designed to connect current students, alumni, academic staff, and administrators of the Department of Computer Engineering, University of Peradeniya. This report documents the architectural decisions, implementation details, cloud deployment strategy, and research findings for the DECP mini project, developed as part of CO528 – Applied Software Architecture.

DECP is built on a Cloud-Native, Domain-Driven Microservices Architecture using Apache Kafka as the event streaming backbone. The platform delivers eight core functional modules: User Management, Social Feed, Jobs & Internships, Events & Announcements, Research Collaboration, Messaging, Notifications, and Analytics. Each module is encapsulated as an independent microservice, satisfying Service-Oriented Architecture (SOA) principles with clearly defined APIs, isolated databases, and asynchronous event-driven communication.

The technical stack combines NestJS (TypeScript) for backend services, React (TypeScript) for the web client, and a mobile client consuming the same backend APIs through a unified Kong API Gateway. Infrastructure is containerised with Docker and designed for cloud deployment on AWS using Kubernetes.

2. Project Overview & Objectives

2.1 Objective

The primary objective is to design and build a real-world inspired department engagement platform for current students and alumni. The platform enables users to connect, share posts, apply for jobs and internships, collaborate on research, and participate in departmental events — accessible through both web and mobile clients.

The project emphasises architectural design, modularity, and integration over full feature completeness. The core focus areas are:

- Architecture Design – Correctness and clarity of SOA, enterprise, and product modular diagrams
- Modular Services – Each module is an independently deployable microservice
- Integration – Event-driven communication via Apache Kafka
- Cloud Deployment – Backend and database hosted on cloud infrastructure
- Web + Mobile Clients – Functional clients consuming unified backend APIs
- Quality Attribute Justification – Design decisions justified against non-functional requirements

2.2 Scope

In Scope:

- Core modules: User Management, Feed, Jobs, Events, Messaging, Notifications, Analytics, Research Collaboration
- Web client (React) and Mobile client consuming the same APIs
- Local development environment with Docker Compose
- Cloud-ready deployment design (AWS/Kubernetes)
- Event-driven notifications using Apache Kafka

Out of Scope:

- Full AI-based recommendation engine (planned as future extension)
- Third-party payment integration
- Full-scale enterprise HR management beyond job and application management

3. Functional Requirements

The following table summarises the eight core functional modules and their key capabilities:

Module	Key Features	Actors
User Management	Register/Login, profile management, role-based access (Student, Alumni, Admin), bulk CSV provisioning, Google OAuth	All users, Admin
Feed & Media Posts	Create text posts, image/video upload via MinIO, like, comment, share functionality, cursor feed	Students, Alumni
Jobs & Internships	Post job/internship listings, apply via web/mobile, resume upload (PDF), application lifecycle management	Alumni (post), Students (apply), Admin
Events & Announcements	Create department events/workshops, RSVP system, event notifications, admin announcements	Admin, All users
Research Collaboration	Create research projects, invite collaborators, share documents, manage project memberships	Students, Alumni
Messaging	Direct messaging (1-to-1), group chat, real-time WebSocket communication	Students, Alumini
Advanced Notifications	Event-driven push and email notifications, in-app alerts triggered by Kafka consumer events	All users
Analytics Dashboard	Active user tracking, popular post metrics, job application statistics, engagement analytics	Admin

4. Non-Functional Requirements (Quality Attributes)

Quality Attribute	Requirement	Architectural Decision
Scalability	Handle thousands of concurrent users and a growing alumni base	Kubernetes auto-scaling, Kafka horizontal partitioning, stateless microservices
Availability	The system must operate continuously with minimal downtime	Containerised services with health/readiness probes, managed cloud databases
Maintainability	Modular design enabling independent updates and feature additions	Database-per-service, bounded contexts via DDD, NestJS modular architecture
Security	Secure authentication, authorization, and data protection	JWT (Access + Refresh tokens), RBAC, Kong API Gateway, HTTPS/TLS, Helmet middleware
Performance	Low latency for feed, messaging, and notifications	Redis caching, cursor-based pagination, Kafka async decoupling, indexed MongoDB/PostgreSQL
Observability	Full visibility into system health and performance	Prometheus metrics, Grafana dashboards, ELK centralized logging, Open Telemetry tracing

5. Architecture Design

5.1 Enterprise Architecture

The enterprise architecture describes the system at a business level, showing actors, system modules, high-level integration, and departmental workflow. DECP follows a Domain-Driven Microservices Architecture where each module represents a bounded context and cross-service communication is handled asynchronously via Apache Kafka.

Actors and their roles within the enterprise:

Actor	Responsibilities
Student	Create posts, comment, like, view jobs, apply for opportunities, participate in events, and direct messaging
Alumni	All student capabilities, plus post jobs/internships, create research collaborations, and mentor students
Admin	Manage users, moderate content, handle events, manage notifications, bulk user provisioning, and analytics
Academic Staff	Participate in research collaboration, post announcements, manage events

The architectural structure flows as follows: Users interact through Web or Mobile clients, which communicate exclusively with the Kong API Gateway. The gateway routes request the appropriate microservices. Services communicate synchronously via REST for immediate responses and asynchronously via Kafka for decoupled event propagation. Each service maintains its own isolated database, following the Database-per-Service pattern. The Notification and Analytics services are pure Kafka consumers, reacting to events without initiating their own business logic.

5.2 Service-Oriented Architecture (SOA)

DECP implements a strict SOA model where each of the eight bounded contexts is an independently deployable microservice with well-defined REST APIs. Services do not share databases or internal implementations.

The eight core microservices and their bounded contexts:

Service	Bounded Context	Primary Responsibility	Database
Identity Service	Identity & Access	Authentication, JWT, RBAC, user profile management, user events	PostgreSQL + Redis
Engagement Service	Social Feed	Posts, media uploads, likes, comments, shares	MongoDB + MinIO
Career Service	Jobs & Applications	Job/internship postings, applications, and resume management	MongoDB + MinIO
Event Service	Events & RSVP	Department events, workshops, RSVP system	MongoDB
Collaboration Service	Research	Research projects, collaborators, document sharing	MongoDB + MinIO
Messaging Service	Real-time Chat	Direct messages, group chats, WebSocket connections	MongoDB
Notification Service	Notifications	Kafka consumer; email and in-app push notifications	MongoDB
Analytics Service	Analytics	Kafka consumer; aggregated metrics and dashboard data	MongoDB

5.3 Event-Driven Architecture (Apache Kafka)

Apache Kafka serves as the central event backbone of the DECP platform, enabling asynchronous, decoupled communication between services. This design eliminates direct service-to-service dependencies for non-critical paths, allowing each service to evolve independently.

5.3.1 Kafka Topic Design

Topics are designed by business domain rather than per service, following enterprise best practices for scalability and observability:

Topic	Domain	Key Events
identity.events	Identity & Access	identity.user.created, identity.user.updated, identity.user.role_changed
engagement.events	Social Feed	engagement.post.created, engagement.post.liked, engagement.post.commented
career.events	Career	career.job.posted, career.job.published, career.application.submitted, career.application.reviewed
event.events	Events	event.created, event.rsvp
collaboration.events	Research	project.created, collaborator.invited
message.events	Messaging	message.sent
notification.events	Notifications	notification triggered events
analytics.events	Analytics	Aggregated statistics events

5.3.2 Standard Event Schema

All Kafka events follow a standardized envelope schema to support duplication, distributed tracing, versioning, and audit capability:

```
{ "eventId": "uuid",
  "eventType": "identity.user.created",
  "eventVersion": "1.0",
  "timestamp": "2026-01-01T12:00:00Z",
  "producer": "identity-service",
  "actor" : "uuid"
  "correlationId": "uuid",
  "data": { ... } }
```

5.3.3 Example Event Flow: Job Application

- User submits job application via web/mobile client
- Career Service validates and persists the application to MongoDB
- Career Service publishes career.application.submitted to Kafka
- Notification Service consumes event and sends email + in-app notification to applicant and job poster
- Analytics Service consumes events and increments application statistics

No direct coupling between Career, Notification, or Analytics services. New services (e.g., AI Recommendation) can subscribe to existing topics without modifying any existing service.

5.4 Communication Model

DECP uses a hybrid communication model:

Type	Protocol	Used For	Examples
Synchronous	REST (HTTP/HTTPS)	Immediate client-facing responses	Login, fetch feed, apply for a job, RSVP event
Asynchronous	Apache Kafka	Decoupled cross-service events	Notifications, analytics updates, and role change propagation
Real-time	WebSocket	Live messaging	Direct messages, group chat

5.5 Web-Oriented Architecture

The React web client is a single-page application (SPA) that consumes all backend services exclusively through the Kong API Gateway. The client implements:

- Authentication flow with JWT access and refresh token management
- Role-based routing (different views for Student, Alumni, Admin)
- Feed, Jobs, Events, and Analytics UIs
- Real-time messaging via WebSocket connection to the Messaging Service
- File uploads (images, resumes, documents) directly to MinIO-backed endpoints

5.6 Mobile Architecture

The mobile client consumes the identical REST API endpoints exposed through the Kong API Gateway, ensuring full feature parity with the web client for core modules. The same JWT-based authentication strategy applies, with refresh token storage handled securely on the device. Core features implemented on mobile:

- Authentication and user profile management
- Social feed with post creation and interaction
- Job listings and application submission
- Event listings and RSVP functionality

6. Product Architecture

6.1 Modular Design

The product follows a modular architecture with clear separation between core modules (required for MVP), extended modules (full feature set), and infrastructure services (cross-cutting concerns). This design supports independent development, testing, and deployment of each module.

Category	Modules
Core Modules	Identity Service, Engagement Service, Career Service, Event Service
Extended Modules	Collaboration Service, Messaging Service, Notification Service, Analytics Service
Infrastructure Services	Kong API Gateway, Apache Kafka, Redis Cache, MinIO Object Storage
Observability Stack	Prometheus, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana), OpenTelemetry
Future / Optional	AI Recommendation Engine, Search Service (Elasticsearch-based), Payment Integration

6.2 Reusable Components

A shared monorepo library (/shared/event-bus/) provides reusable components across all microservices:

- BaseEvent interface – Standardised event envelope for all Kafka messages
- Kafka Producer wrapper – Unified publish function with automatic serialisation
- Kafka Consumer factory – Configurable consumer group creation
- Correlation ID middleware – Request tracing across services
- OpenTelemetry integration – Distributed trace propagation

7. Technology Stack

Layer	Technology	Justification
Frontend (Web)	React + TypeScript + Vite + MUI	Component-based SPA with strong typing; fast HMR with Vite; professional UI with Material UI
API Gateway	Kong Gateway 3.6	Enterprise-grade API management; JWT validation; rate limiting; centralised routing
Backend Framework	NestJS (Node.js + TypeScript)	Structured modular architecture; built-in microservice and Kafka support; enterprise-ready
Event Streaming	Apache Kafka (KRaft mode)	High-throughput async messaging; event replay; decoupled service communication
Identity Database	PostgreSQL 15	ACID-compliant relational DB; strong for user data, roles, and relational queries
Other Service Databases	MongoDB	Flexible schema for posts, jobs, events, messages; horizontal scaling
Caching	Redis 7	Session caching; OTP storage; rate limiting; online status tracking; event idempotency
Object Storage	MinIO (S3-compatible)	Profile images, resumes, event media; local S3 simulation; cloud-portable abstraction
Containerization	Docker + Docker Compose	Service isolation; reproducible environments; Kubernetes-ready images
Orchestration (Future)	Kubernetes (EKS)	Auto-scaling; pod management; production deployment
Metrics	Prometheus + Grafana	API response times; error rates; Kafka consumer lag; DB performance monitoring
Logging	ELK Stack	Centralised structured logs; searchable by userId, correlationId; distributed debugging
Tracing	OpenTelemetry	End-to-end request tracing across microservices; latency bottleneck identification
CI/CD	GitHub Actions	Automated build; Docker image versioning; deployment pipeline

8. Implementation Details

8.1 Identity Service

The Identity Service is the authentication and authorisation authority for the entire DECP platform. It uses PostgreSQL as its primary database with Redis for caching and session management.

8.1.1 Database Schema (PostgreSQL)

The Identity Service manages a rich relational schema designed for extensibility and performance:

Table	Purpose	Key Fields
users	Core user identity record	id (UUID), reg_number, email, google_id, first_name, last_name, role (ENUM), profile_pic, headline, bio
social_links	User social media profiles	id, user_id (FK), platform (ENUM: LinkedIn, GitHub, etc.), url
projects	User portfolio projects	id, user_id (FK), title, start_date, end_date, description, link
experiences	Work experience history	id, user_id (FK), title, emp_type (ENUM), company, start_date, end_date
educations	Academic qualifications	id, user_id (FK), institution, degree, field_of_study, grade
publications	Academic publications	id, user_id (FK), title, journal, published_date, link

8.1.2 Redis Usage

Redis is used for the following purposes within the Identity Service:

- Online Status Tracking: Key online:user:<userId> with 2-minute TTL; users ping every 60 seconds to maintain presence without constant DB writes
- OTP Storage: Temporary one-time passwords with short expiry
- Rate Limiting: Per-user and per-IP request throttling
- Event Idempotency: Processed Kafka eventId storage to prevent duplicate notification delivery

8.1.3 Authentication Flow

- Client authenticates via Kong Gateway to Identity Service
- Identity Service validates credentials and issues JWT Access Token (15-minute expiry) and Refresh Token (7-day expiry)
- JWT payload includes sub (userId), email, role, permissions, iat, exp

- All subsequent requests carry the Access Token; Identity Service validates on each protected route
- Google OAuth 2.0 supported via google-auth-library for single sign-on

8.1.4 Role-Based Access Control (RBAC)

Three roles are defined: ADMIN, STUDENT, and ALUMNI. Role guards are implemented as NestJS decorators applied at the controller level. Key RBAC rules:

- Students: Can create posts, like/comment, apply for jobs, join events, direct message
- Alumni: All student capabilities plus post jobs/internships and create research projects
- Admin: Full platform management; bulk user provisioning; content moderation; analytics access
- Self-Demotion Protection: Admins cannot change their own role via the bulk role update endpoint

8.1.5 Bulk User Provisioning

The Admin dashboard supports bulk CSV import for new student cohorts, following a four-phase workflow:

1. Downloadable standardised CSV template
2. Dry-run validation endpoint returning a preview report of valid/invalid rows
3. Batch insertion using Prisma createMany in a single transaction
4. Single BatchUsersProvisioned Kafka event triggering automated welcome emails via the Notification Service. This reduces admin onboarding effort from hours of manual data entry to approximately 45 seconds.

8.1.6 Kong API Gateway Integration

The Identity Service is registered in Kong as a named upstream service. All /identity/* routes are proxied through Kong on port 8000. JWT validation is performed within NestJS guards; Kong can optionally be configured to handle centralised JWT validation in future.

8.2 Engagement Service

The Engagement Service manages the social feed functionality. It is built with Express + TypeScript (Node.js) and uses MongoDB as its database with MinIO for media storage.

8.2.1 Core Features

- Post creation with text content and media attachments (images/videos via MinIO)
- Like system with atomic MongoDB updates
- Comment system with nested threading support
- Share functionality with reference to original post
- Cursor-based pagination for infinite scroll feed (sorted by _id DESC)

8.2.2 MongoDB Collections

```
"posts": {
   "_id": "mongo_object_id",
   "authorId": "mongo_object_id",
   "content": "string",
   "mediaUrls": ["string"],
   "likesCount": 0,
   "commentsCount": 0,
   "sharesCount": 0,
   "createdAt": "date",
   "updatedAt": "date"
}

"comments": {
   "_id": "mongo_object_id",
   "postId": "mongo_object_id",
   "authorId": "mongo_object_id",
   "content": "string",
   "createdAt": "date"
}

"likes": {
   "_id": "mongo_object_id",
   "postId": "mongo_object_id",
   "userId": "mongo_object_id",
   "createdAt": "date"
}
— unique compound index (postId, userId)
```

8.2.3 Kafka Events Published

- engagement.post.created — consumed by Notification and Analytics Services
- engagement.post.liked — consumed by Analytics Service
- engagement.post.commented — consumed by Notification and Analytics Services

8.3 Career Service

The Career Service manages the full job and internship lifecycle, from posting through application review. It uses MongoDB for job and application data and MinIO for resume storage.

8.3.1 Domain Models

```
"Job": {
   "_id": "mongo_object_id",
   "title": "string",
   "description": "string",
   "location": "string",
   "employmentType": "ENUM",
   "department": "string",
   "salaryRange": {
       "min": 0,
       "max": 0
   },
   "status": "DRAFT|PUBLISHED|CLOSED",
   "postedBy": "mongo_object_id",
```

```
    "applicationCount": 0,
    "createdAt": "date",
    "updatedAt": "date"
}

"Application": {
    "_id": "mongo_object_id",
    "jobId": "mongo_object_id",
    "applicantId": "mongo_object_id",
    "resumeUrl": "string",
    "coverLetter": "string",
    "status": "SUBMITTED|REVIEWED|REJECTED|ACCEPTED",
    "createdAt": "date",
    "updatedAt": "date"
}
```

8.3.2 Access Control Rules

- Only Alumni and Admin can create and publish job postings
- Only students and alumni can apply for jobs (Alumni can't apply to their own posted jobs)
- Only Admin and the posting Alumni can review applications
- Duplicate application prevention enforced via a unique compound index on (jobId, applicantId)

8.3.3 Resume Upload Strategy

- Resumes stored in MinIO (S3-compatible); only the URL persisted in MongoDB
- File type validation: PDF only; maximum size: 5MB
- MinIO bucket configured with appropriate access policies

8.4 Event Service

The Event Service handles department events, workshops, and the RSVP system. It uses MongoDB as its database.

- Event creation with rich metadata: title, description, date/time, venue, capacity, category
- RSVP system with attendance tracking and capacity enforcement
- Admin announcements as a special event type
- Publishes event.created and event.rsvp events to Kafka for downstream notification and analytics

8.5 Notification Service

The Notification Service is a pure Kafka consumer service with no REST API of its own. It subscribes to all major event topics and triggers appropriate notifications.

8.5.1 Kafka Topics Consumed

- identity.events (user.created, user.role_changed)
- engagement.events (post.created, post.commented)
- career.events (job.posted, application.submitted, application.reviewed)
- event.events (event.created, event.rsvp)
- collaboration.events (project.created, collaborator.invited)
- message.events (message.sent)

8.5.2 Notification Channels

- Email notifications via configured SMTP provider
- In-app notifications stored in MongoDB and delivered via WebSocket
- Push notifications (mobile) via FCM/APNs integration

8.5.3 Idempotency Protection

Each incoming Kafka event is checked against a Redis key (processed:<eventId>) before processing. If the key exists, the event is skipped, preventing duplicate notifications in case of Kafka retries or consumer restarts.

8.6 Analytics Service

The Analytics Service is a pure Kafka consumer that aggregates platform metrics for the Admin dashboard. It consumes all major event types and maintains aggregated statistics in MongoDB. Key metrics tracked:

- Daily and monthly active users (from identity.events)
- Popular posts ranking (from engagement.events)
- Job application statistics and application rates (from career.events)
- Event attendance and RSVP conversion rates (from event.events)
- Engagement metrics: posts per day, comments per post, like rates

8.7 Messaging Service

The Messaging Service provides real-time direct and group messaging using WebSocket connections (Socket.io or native ws). MongoDB stores message history. Kafka integration publishes message.sent events for notification delivery.

- Direct messaging between two users with message history
- Group chat with multiple participants
- WebSocket connection management with online presence indicators (via Redis)
- Message persistence with read status tracking

9. Infrastructure & Cloud Deployment

9.1 Local Development Environment (Docker Compose)

The complete DECP platform runs locally via Docker Compose on a single shared decp-network bridge network, simulating production Kubernetes service discovery. All services communicate using Docker internal DNS (service names as hostnames).

Component	Image	Port	Purpose
Kong API Gateway	kong:3.6	8000 (proxy), 8001 (admin)	Centralised API management and routing
Kong Database	postgres:15	Internal	Kong configuration persistence
Identity Service	Custom NestJS	4000	Authentication and user management
Engagement Service	Custom Express/TS	4001	Social feed microservice
Career Service	Custom NestJS	4002	Jobs and applications
Event Service	Custom NestJS	4003	Events and RSVP
Messaging Service	Custom NestJS	4004	Real-time chat
Notification Service	Custom NestJS	4005	Kafka consumer – notifications
Analytics Service	Custom NestJS	4006	Kafka consumer – metrics
PostgreSQL	postgres:15	5432	Identity Service relational database
MongoDB	mongo:latest	27017	Engagement, Career, Event, Messaging, Analytics databases
Redis	redis:7	6379	Caching, rate limiting, online status, idempotency
Apache Kafka	bitnami/kafka (KRaft)	9092	Event streaming backbone

Component	Image	Port	Purpose
MinIO	minio/minio	9000 (API), 9001 (console)	S3-compatible object storage
Prometheus	prom/prometheus	9090	Metrics collection
Grafana	grafana/grafana	3001	Metrics visualisation
Elasticsearch	elasticsearch:8.11	9200	Log indexing and search
Kibana	kibana:8.11	5601	Log visualisation dashboard

9.2 Cloud Deployment Architecture (AWS)

The DECP platform is designed as a cloud-native application ready for deployment on AWS. The target architecture leverages managed services for reliability and reduced operational overhead:

Component	AWS Service	Justification
DNS	Route 53	Global DNS with health checks and failover routing
CDN	CloudFront	Static asset delivery; reduced latency for global users
API Gateway / Ingress	NGINX Ingress + Kong on EKS	Centralised request routing with JWT validation and rate limiting
Container Orchestration	Amazon EKS (Kubernetes)	Auto-scaling, pod management, rolling deployments, health probes
Microservices	EKS Pods (Docker images)	Each microservice in isolated pod; horizontal pod autoscaler
Event Streaming	Amazon MSK (Managed Kafka)	Fully managed Kafka; multi-AZ replication; enterprise SLA
Relational DB (Identity)	Amazon RDS (PostgreSQL)	Managed PostgreSQL; automated backups; Multi-AZ failover
Document DB (Others)	MongoDB Atlas or DocumentDB	Fully managed MongoDB-compatible; horizontal sharding
Cache	Amazon ElastiCache (Redis)	Managed Redis cluster; automatic failover
Object Storage	Amazon S3	Unlimited scalable object storage; migrated from MinIO
Monitoring	CloudWatch + Prometheus/Grafana	Infrastructure and application-level observability
CI/CD	GitHub Actions + ECR	Automated Docker image build, push to ECR, deploy to EKS

9.2.1 Scalability Considerations

- Stateless microservices: All services designed without local state; session data in Redis; horizontal scaling via pod replicas
- Kafka horizontal partitioning: Topics partitioned across multiple brokers; consumer groups scale consumers independently
- Database read replicas: PostgreSQL and MongoDB configured with read replicas to distribute query load
- API Gateway rate limiting: Kong enforces per-user and per-endpoint rate limits to protect downstream services
- Auto-scaling: Kubernetes HPA (Horizontal Pod Autoscaler) scales pods based on CPU and custom Prometheus metrics

10. Observability & Production Readiness

10.1 Health & Readiness Probes

Every microservice exposes standardised health and readiness endpoints for Kubernetes integration:

- GET /health — Liveness probe: always returns 200 if service process is running
- GET /ready — Readiness probe: returns 200 only when all dependencies (database, Kafka) are connected; returns 503 otherwise
- GET /metrics — Prometheus scrape endpoint returning service metrics in Prometheus text format

10.2 Metrics (Prometheus + Grafana)

Each service exposes application-level metrics via prom-client:

- HTTP request counter: Total requests by endpoint and status code
- Request duration histogram: P50, P95, P99 latency per endpoint
- Domain-specific counters: e.g., career_applications_submitted_total, engagement_posts_created_total
- Kafka consumer lag: Offset lag per consumer group for event processing health

10.3 Centralised Logging (ELK Stack)

All services use structured JSON logging (Pino) with mandatory fields: timestamp, level, service, correlationId, userId (when available), and message. Logs are forwarded to Elasticsearch via Logstash and visualised in Kibana. This enables cross-service log correlation using correlationId for debugging distributed request flows.

10.4 Distributed Tracing (OpenTelemetry)

OpenTelemetry SDK is integrated into each service. Every incoming HTTP request generates a trace with a unique traceId. Kafka events carry the correlationId to link event-driven flows. OTEL spans wrap all database calls and external service interactions, recording exceptions and key attributes (userId, jobId, etc.). This enables end-to-end tracing from client request through API Gateway, across microservices, and into Kafka event consumption.

10.5 Error Handling Strategy

- Global error handlers in each NestJS service return a standardized error response format
- Kafka dead-letter topics: Failed consumer events after retry exhaustion are moved to DLT topics for manual inspection
- Retry mechanisms: Kafka consumer retries with exponential backoff before dead-lettering
- Circuit breaker pattern: Future enhancement for synchronous REST calls between services

11. Security Architecture

Security Control	Implementation	Purpose
Authentication	JWT (Access: 15min, Refresh: 7 days) + Google OAuth 2.0	Stateless, scalable authentication without server-side session storage
Authorisation	RBAC via NestJS guards and decorators	Fine-grained, role-based access control per endpoint
API Security	Kong Gateway: rate limiting, JWT validation, HTTPS termination	Centralised security enforcement at the network boundary
HTTP Security Headers	Helmet middleware on all NestJS services	Protection against XSS, clickjacking, MIME sniffing
Input Validation	NestJS class-validator DTOs on all endpoints	Prevents injection attacks; strips unexpected fields
File Upload Security	File type validation (whitelist), 5MB size limit, MinIO isolation	Prevents malicious file uploads and storage abuse
Transport Security	HTTPS/TLS everywhere (Kong handles termination)	Data in transit encryption; prevents man-in-the-middle attacks
Secret Management	Environment variables; no hardcoded secrets in code	Secure credential management; Docker secrets for production
Role Escalation Protection	Admin self-demotion guard; DTO field stripping	Prevents accidental or malicious privilege modification

12. Research Findings: Platform Analysis & Improvement Proposals

12.1 Analysis of Real-World Platforms

As required by the project brief, the team analysed real-world platforms including LinkedIn and Facebook to understand their architectural approaches and identify features relevant to DECP.

Platform	Architecture Pattern	Key Architectural Features
LinkedIn	SOA evolving to microservices	Kafka for activity feeds and messaging; REST APIs; Espresso (key-value store) for profiles; Samza for stream processing; Voldemort for distributed storage
Facebook	Service mesh microservices	TAO graph store for social graph; Haystack for photo storage; Messenger protocol buffer APIs; React Native for mobile consistency
Slack	Event-driven microservices	WebSocket gateway for real-time; Kafka for message fanout; Redis for presence; separate pub/sub from message storage

12.2 Feature Gaps Identified

Compared to LinkedIn and Facebook, the following features are either absent or simplified in DECP that represent opportunities for improvement:

Gap Area	Current DECP State	Proposed Improvement
Search & Discovery	No dedicated search service	Elasticsearch-based Search Service consuming Kafka events to index posts, jobs, users & projects in real-time
AI Recommendations	Not implemented (out of scope)	ML-based job and connection recommendation engine as a new Kafka consumer service; no changes to existing services required
Content Moderation	Manual admin moderation only	Automated content moderation using text classification models integrated into the Engagement Service pipeline
Skill Endorsements	Not implemented	LinkedIn-style skill tagging and peer endorsement system within the Identity/Profile domain
Alumni Verification	Role assignment by admin only	Automated alumni verification via university email domain or student ID validation workflow
Feed Algorithm	Chronological feed only	Engagement-weighted feed ranking using Analytics Service data; user interest modelling
Video Streaming	Video upload to MinIO	Adaptive bitrate streaming (HLS/DASH) via CDN for large video content; transcoding pipeline
Internship Matching	Manual job search only	Automated internship matching based on student skills and alumni job requirements using ML scoring

12.3 Architectural Improvements Proposed

12.3.1 AI Integration Roadmap

Because DECP uses an event-driven Kafka backbone, AI and ML services can be introduced without modifying any existing service. New AI services simply subscribe to existing topics:

- Recommendation Service: Subscribes to identity.events, engagement.events, career.events to build user interest models
- Content Moderation Service: Integrates into Engagement Service as a pre-publish validation step
- Smart Notifications Service: Replaces or augments Notification Service with ML-based notification relevance scoring and delivery timing optimisation

12.3.2 Schema Registry for Kafka

Production-scale Kafka deployments require a Schema Registry (e.g., Confluent Schema Registry with Avro) to enforce event schema contracts between producers and consumers. This prevents schema-breaking changes from silently corrupting consumer logic across services.

12.3.3 Service Mesh

As the number of microservices grows, introducing a service mesh (e.g., Istio or Linkerd) on Kubernetes would provide: automatic mTLS between services, advanced traffic management (canary deployments, A/B testing), and built-in distributed tracing without code-level instrumentation.

13. CI/CD & DevOps Strategy

The CI/CD pipeline is implemented using GitHub Actions with the following stages:

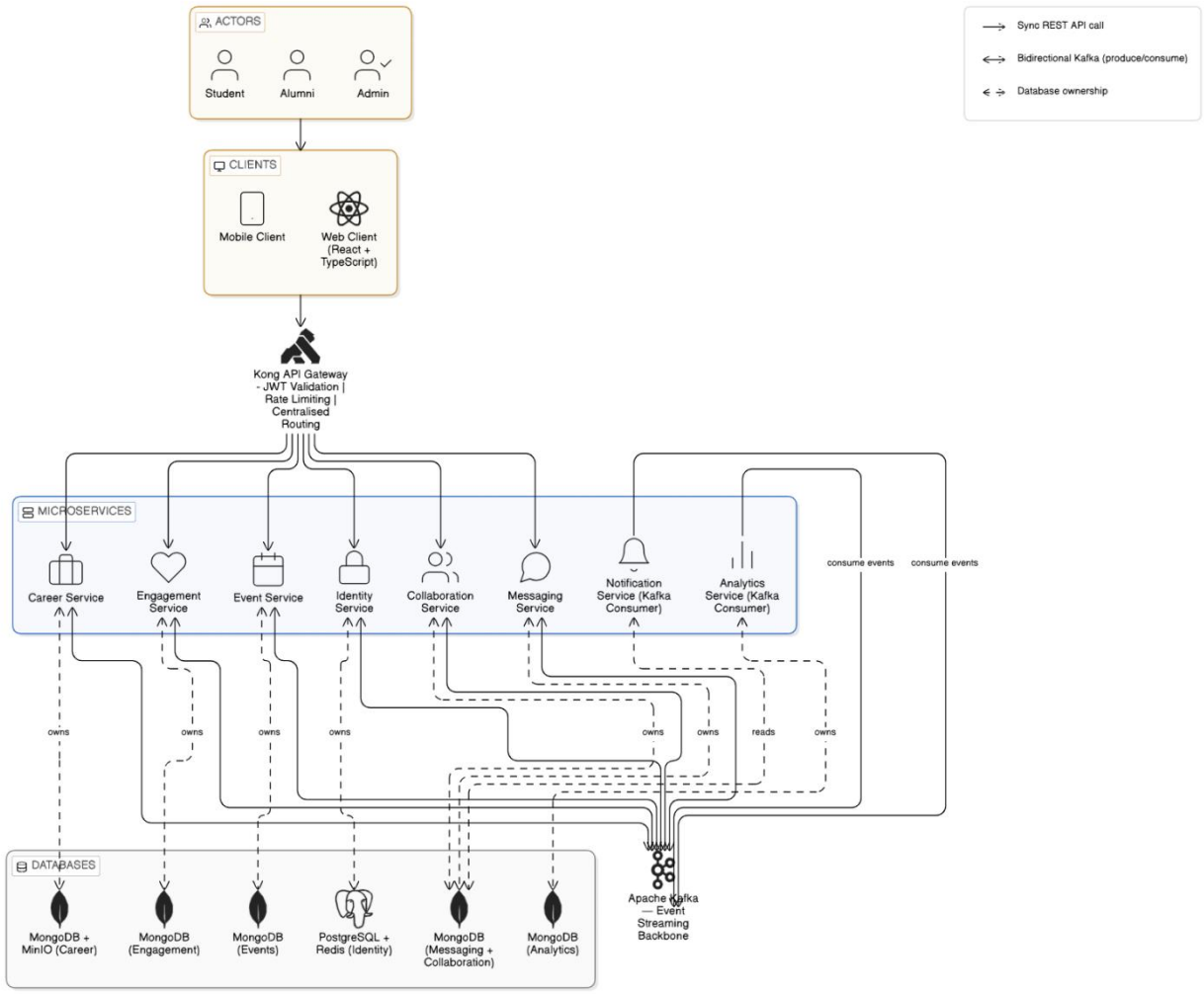
Stage	Action	Trigger
Build	npm install, TypeScript compile, unit tests	On every push to feature branches
Docker Build	Build Docker image for changed service	On pull request merge to main
Image Push	Push versioned image to Amazon ECR (or Docker Hub)	On successful build
Deploy	kubectl apply with updated image tag to Kubernetes cluster	On main branch after image push
Health Check	Verify pod readiness probes pass post-deployment	Post-deploy automated verification

14. Architecture Diagrams

The following architecture diagrams describe the overview of the system.

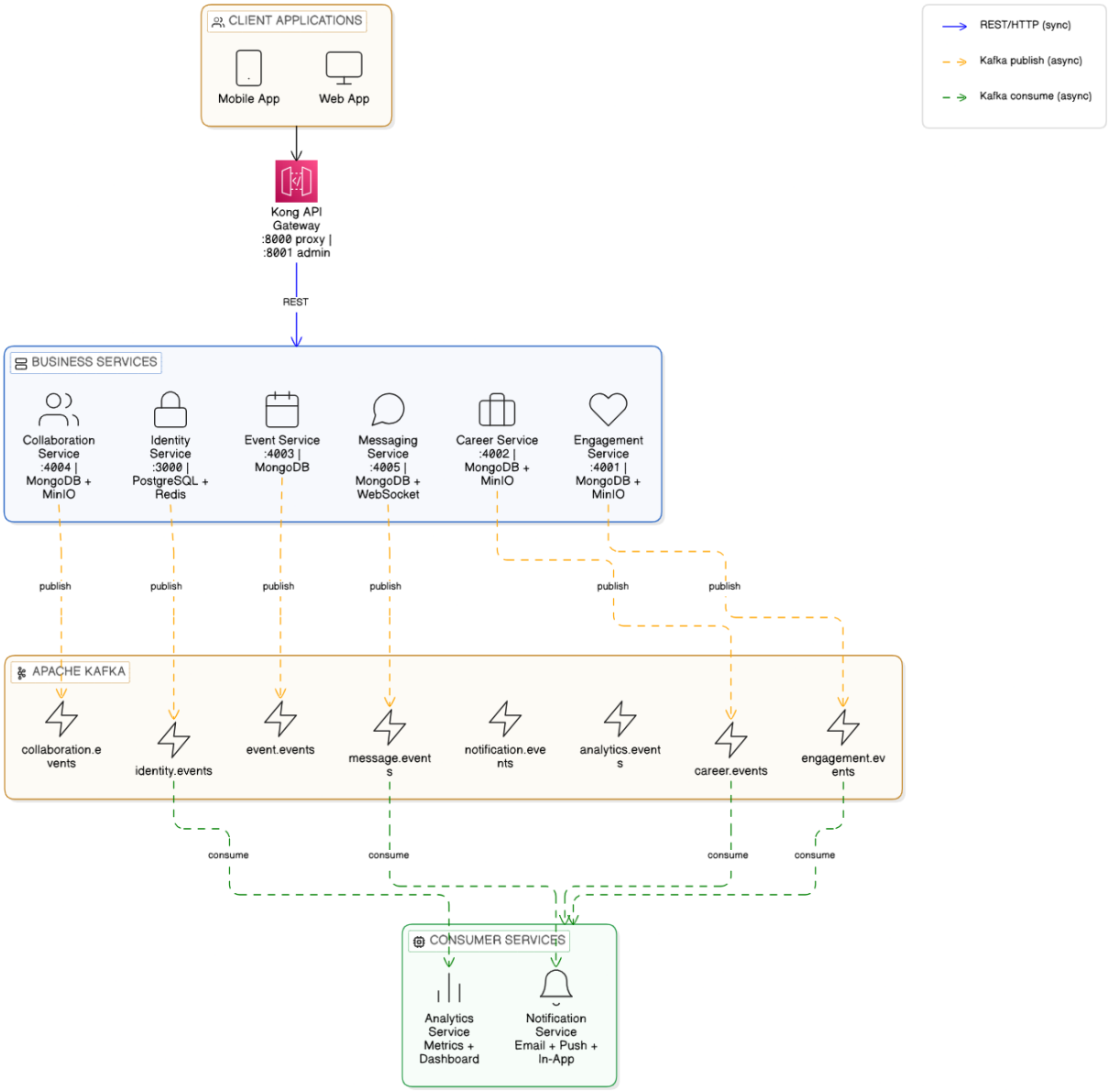
14.1 Enterprise Architecture Diagram

High-level business view showing actors (Student, Alumni, Admin), system modules, client layers, API Gateway, microservices layer, Kafka event bus, and database tier



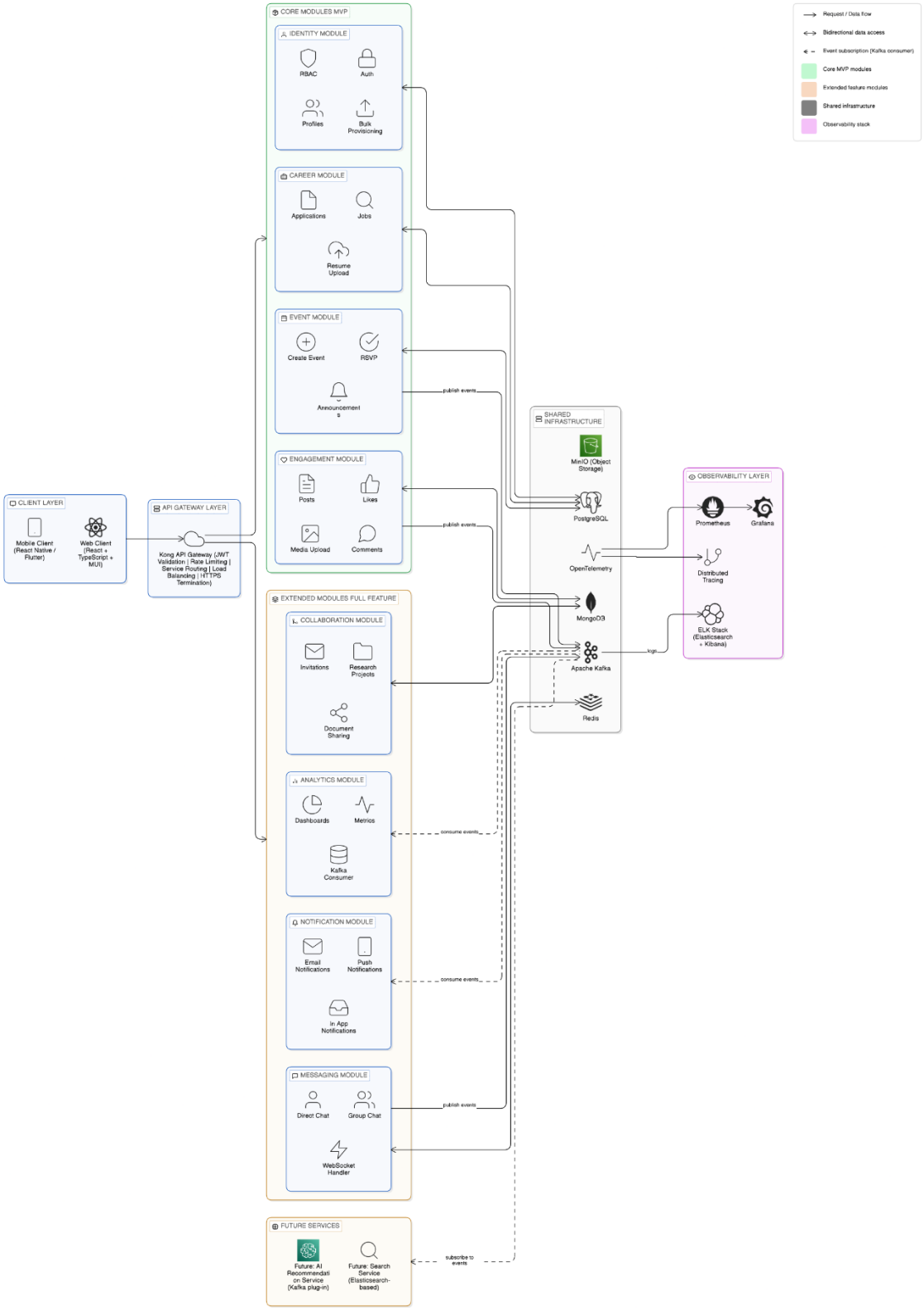
14.2 SOA/Microservice Architecture Diagram

Technical service-level view showing all 8 microservices, their REST API interactions, Kafka event flows, and service independent boundaries



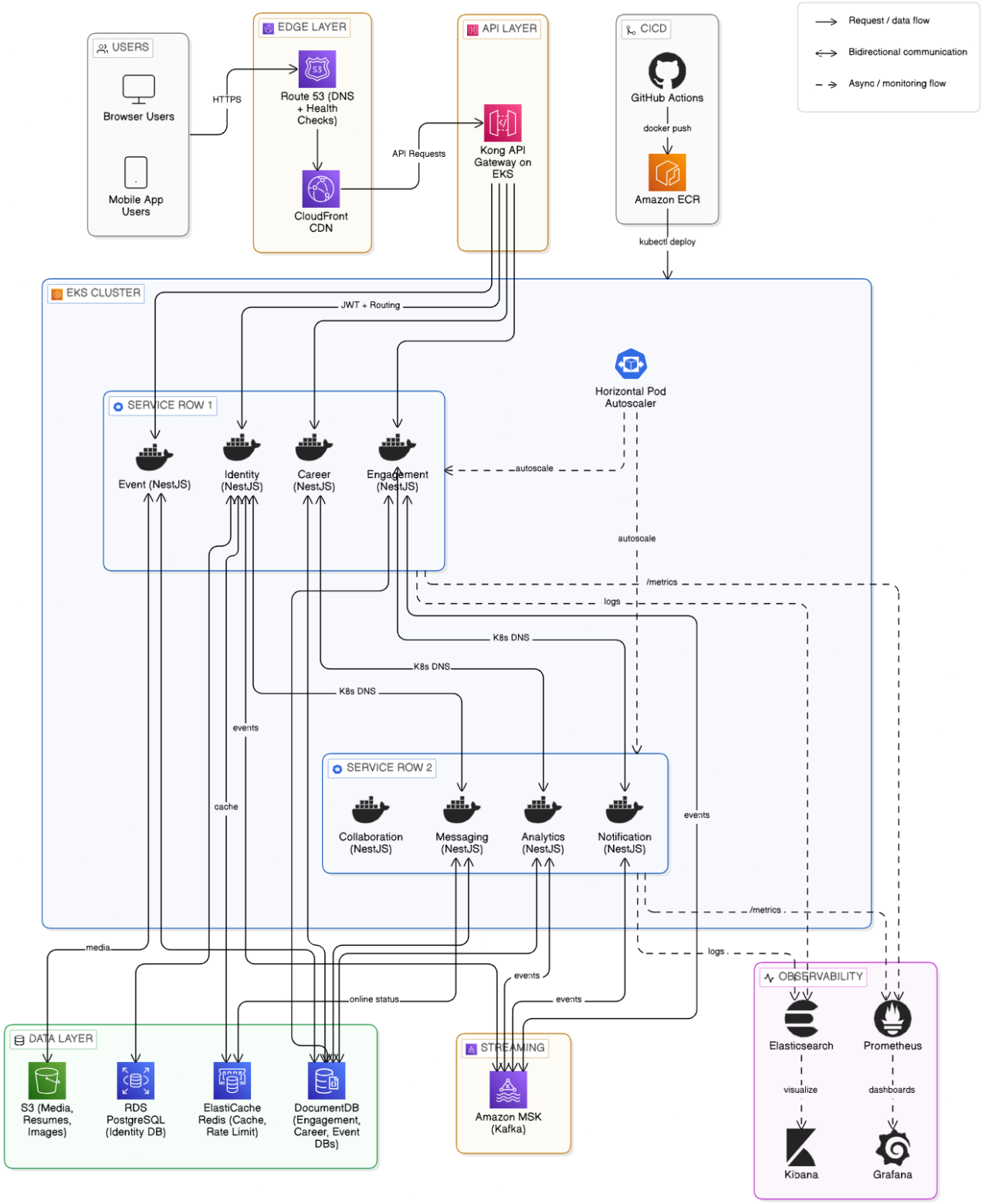
14.3 Product Modularity Diagram

Core vs. extended vs. infrastructure modules; optional features; reusable shared components; maintainability plan



14.4 Deployment Architecture Diagram

Cloud deployment on AWS: Route 53, CloudFront, Kong on EKS, MSK Kafka, RDS PostgreSQL, MongoDB Atlas, ElastiCache Redis, S3, Prometheus/Grafana, ELK



15. Conclusion

The Department Engagement & Career Platform (DECP) successfully demonstrates the application of advanced software architecture principles in a real-world-inspired context. The platform's Cloud-Native Domain-Driven Microservices Architecture with Apache Kafka event streaming provides a robust, scalable, and maintainable foundation for university engagement and career management systems.

The architectural decisions made throughout the project directly satisfy the project requirements across SOA, Web-Oriented Architecture, Mobile Architecture, Cloud Architecture, Enterprise Architecture, and Product Architecture dimensions. The Database-per-Service pattern ensures true service independence, while the shared Kafka event bus enables loose coupling and independent evolution of each bounded context.

The research findings from LinkedIn and Facebook demonstrate that DECP's architecture aligns with industry best practices and provides a clear upgrade path, particularly for AI-powered features, without requiring structural changes to existing services, which is a hallmark of well-designed event-driven architecture.

The observability stack (Prometheus, Grafana, ELK, OpenTelemetry) ensures production-grade visibility, and the security architecture (JWT, RBAC, Kong, Helmet, input validation) meets enterprise security standards. This platform is positioned as a sustainable, extensible foundation that can grow with the department's needs for years to come.

16. Appendix: Reference Links

- **Project Repo:** https://github.com/cepdnaclk/PeraCom-DECP
- **Project Page:** https://cepdnaclk.github.io/PeraCom-DECP/
- **Demonstrations:** https://drive.google.com/drive/folders/1bKYh8Yxp8dpBnpTZEk3IQ20JCg8Kw8Ri?usp=sharing

***

End of Report

CO528 Applied Software Architecture | Department of Computer Engineering | University of Peradeniya